GIT & GITHUB

A dark-mode beginner's field guide • concepts, commands, and real workflows

What you'll learn: version control, repositories, commits, branches, merging, remotes, GitHub, pull requests, conflicts, collaboration, and safe habits. Code blocks are intentionally high-contrast for readability.

1. Git vs GitHub

Git is a version-control tool that tracks changes to files on your computer. It lets you review history, create branches, and return to earlier versions. **GitHub** is an online platform for hosting Git repositories and collaborating through issues, pull requests, reviews, and automation.

Term	Plain-English meaning	Example
Repository (repo)	A project plus its tracked history.	A Python or web app folder
Working tree	Files as they currently appear on disk.	You edit app.py
Staging area	The changes selected for the next commit.	git add app.py
Commit	A saved snapshot with a message.	git commit -m "Add login"
Branch	An independent line of development.	feature/login
Remote	A named copy hosted elsewhere.	origin on GitHub

Install and check

```
git --version
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --list
```

Set your name and email once on your machine. Use an email address you are comfortable associating with your commits. These settings label commits; they do not log you into GitHub.

2. Your first local Git repository

A repository lets Git track changes in a folder. The standard loop is: edit files → stage selected changes → commit a snapshot.

Create a repository

```
mkdir hello-git
cd hello-git
git init
git status
```

Create a file such as `README.md`, add a line of text, then run:

```
git status
git add README.md
git status
git commit -m "Add project README"
git log --oneline
```

The three areas to remember

Area	What it means	Typical command
Working tree	Your current edits.	edit a file
Staging area	Changes chosen for next commit.	git add file
Repository history	Saved commits.	git commit -m "message"

Useful everyday commands

```
git status
git diff
git diff --staged
git log --oneline
git show
git restore file.txt
git restore --staged file.txt
```

Meaning: `diff` shows unstaged edits; `diff --staged` shows staged changes; `show` displays a commit. `restore file.txt` discards unstaged edits to that file—use carefully. `restore --staged` removes a file from staging without deleting its edits.

Before committing, run **git status** and review the diff. A commit should represent one understandable change.

3. Branches and merging

A branch is a lightweight pointer to a line of commits. Branches let you develop a feature without mixing unfinished work into the main branch.

Create and switch to a feature branch

```
git branch
git switch -c feature/readme-update
# edit README.md
git status
git add README.md
git commit -m "Improve README"
git switch main
git merge feature/readme-update
```

Your default branch may be called `main`; older projects may use `master`. Use the actual branch name shown by `git branch`. Merging brings changes from one branch into the current branch.

Resolve a merge conflict

A conflict occurs when Git cannot automatically combine edits to the same part of a file. Git marks the conflicting section in the file.

```
<<<<<<< HEAD
Text from the current branch
=======
Text from the branch being merged
>>>>>>> feature/readme-update

# Edit the file to keep the correct final version, then:
git status
git add README.md
git commit
```

Remove all conflict markers after deciding what the final content should be. Do not blindly keep one side unless you know it is correct.

Undo carefully

Command	Effect / caution
git restore file	Discard unstaged edits to a file.
git restore --staged file	Unstage a file; keep its edits.
git revert COMMIT	Create a new commit that reverses a prior commit.
git reset --soft HEAD~1	Move branch back one commit; keep changes staged.
git reset --hard HEAD~1	Discard tracked changes and move branch; destructive.

Avoid **git reset --hard** and force-pushing until you understand the consequences. They can discard work or rewrite shared history.

4. GitHub: connect, push, and pull

Git is the version-control system; GitHub hosts repositories online. A remote connects your local repository to a hosted repository.

Connect an existing local repo

Create an empty repository on GitHub first. Then copy its HTTPS or SSH URL and use it below:

```
git remote add origin https://github.com/USERNAME/REPO.git
git branch -M main
git push -u origin main
```

Replace the example URL with your real repository URL. `origin` is just the conventional name for the remote. `-u` sets the upstream so future pushes and pulls can usually use the shorter commands.

Clone an existing GitHub repository

```
git clone https://github.com/OWNER/REPO.git
cd REPO
git remote -v
```

Sync changes

```
git pull
git push
git fetch
git status
```

fetch downloads remote updates without integrating them into your current branch. **pull** fetches and integrates changes (commonly via merge or rebase, depending on configuration). **push** sends your local commits to the remote.

Authentication basics

GitHub generally uses browser-based authentication through credential tools, or SSH keys. A GitHub account password is not accepted for ordinary Git-over-HTTPS authentication. Follow GitHub's official setup flow if prompted; never paste tokens or private keys into chats or commit them into code.

Task	Command
See remotes	git remote -v
Download remote updates	git fetch
Integrate updates	git pull
Upload commits	git push
Set upstream branch	git push -u origin main

5. Pull requests and team collaboration

A **pull request (PR)** is a GitHub proposal to merge changes from one branch into another. It gives teammates a place to review code, discuss changes, run checks, and approve work before merging.

Typical feature workflow

```
git switch main
git pull
git switch -c feature/add-search
# make and test your changes
git status
git add .
git commit -m "Add search feature"
git push -u origin feature/add-search
```

On GitHub, open a pull request from `feature/add-search` into `main`. Explain what changed, why, and how you tested it. After review and required checks, merge using the project's chosen method.

Common collaboration concepts

Concept	What it does
Issue	Tracks a bug, task, question, or idea.
Pull request	Proposes changes and supports review.
Review	Feedback or approval on proposed code.
Actions	Automates tests, builds, releases, and workflows.
README.md	Explains what the project does and how to run it.
.gitignore	Lists files Git should ignore when untracked.

Example .gitignore for a Python project

```
__pycache__/
*.py[cod]
.venv/
.env
.DS_Store
```

Ignoring a file does not remove it from Git if it was already tracked. Never commit API keys, passwords, access tokens, private datasets, or secrets. If a secret is committed, revoke/rotate it; deleting the line in a later commit is not enough.

A useful commit message

```
git commit -m "Fix empty search results"
git commit -m "Add Docker setup instructions"
git commit -m "Update evaluation script"
```

6. Troubleshooting, best practices, and cheat sheet

Common problems

Problem	What to do first
Nothing to commit	Run git status; files may be unchanged or ignored.
Push rejected	Fetch/pull remote changes, integrate them, then push.
Wrong branch	Run git branch --show-current; switch to the intended branch.
Accidentally staged a file	git restore --staged FILE
Merge conflict	Edit conflict markers, test, git add, then finish merge.
Secret committed	Revoke/rotate it immediately; then clean history if needed.

Daily workflow cheat sheet

```
git status                          # See what's changed
git diff                            # Review unstaged edits
git add FILE                        # Stage one file
git add .                           # Stage changes in current directory
git commit -m "Describe change"     # Save a snapshot
git switch -c feature/name          # Create and switch branch
git switch main                     # Switch branch
git pull                            # Get and integrate remote changes
git push                            # Upload commits
git log --oneline                   # Compact history
```

Beginner practice exercise

1) Create a local repository. 2) Add a README and commit it. 3) Create a feature branch and edit the README. 4) Commit the change. 5) Create an empty GitHub repository, connect it as `origin`, and push. 6) Open a pull request and merge it. 7) Clone the repository into a second folder and inspect its history.

Mental model: edit → `git add` (stage) → `git commit` (save locally) → `git push` (publish). Branches isolate work; pull requests make collaboration reviewable.

Commands assume a modern Git version. Some commands depend on your repository's branch names and remote URL. Review destructive commands before running them.